Machine Learning Tutorial

Name: Nuzhat Mukhtar
Student ID: 24158838

Contents

Introduction 1
Problem and Dataset Overview 2
Exploring the Dataset 3
Data Preprocessing 4
Train-Test Split 5
Understanding Entropy 5
Decision Tree Model Training 7
Predictions 8
Model Evaluation 8
Decision Tree Visualization 9
Feature Importance 11
Model Improvement (Depth Testing and Overfitting) 12
Example Prediction 13
Discussion 14
Conclusion 14
Important Links 14
References 15

Introduction

Machine learning is a branch of artificial intelligence in which the computer is made to learn from the data rather than being programmed directly (Mitchell, 1997). Instead of programming the decision-making process step by step, the data is given as input, and the decision-making process is learned. This process is then applied to predict the output of the new data. This makes the application of machine learning useful in the real world where programming the decision-making process is difficult.

One of the forms of machine learning is classification. In classification, the prediction is made in the form of classification. For instance, the classification of emails as ‘spam’ or ‘not spam’ can be done using classification. In classification problems, the output is always in the form of ‘yes’ and ‘no’.

In this tutorial, the Decision Tree model is used. In the decision tree model, the decision-making process is carried out in the form of a flowchart. In the decision tree model, the decision is made in stages by asking questions and answering them accordingly.

The aim of this tutorial is to understand the working of the decision tree model and how it is implemented using Python.

This tutorial is a complete guide on how the decision tree model is implemented using Python.

Problem and Dataset Overview

In this project, we are taking the bank marketing dataset from UCI archives (Repository, 2012). This dataset is loaded using the following code in the Python script:

```
df = pd.read_csv("bank-full.csv")
```

In this dataset, each row represents the customers of the bank. In the dataset, the information of the customers is represented in the form of columns.

The goal of this project is to forecast whether or not a customer will subscribe to a bank product. This information is contained in the column ‘y’, where it will be either ‘yes’ or ‘no’. Since we are trying to forecast two outcomes, it’s a binary classification problem.

In the code, the target variable ‘y’ has been converted into a numerical format as follows:

```
df['y'] = df['y'].map({'yes': 1, 'no': 0})
```

So, the target variable ‘y’ has the following mappings:

```
Yes → 1

No  → 0
```

This conversion is necessary since the machine learning model requires a numerical input.

Exploring the Dataset

After loading the dataset, it needs to be explored to understand the structure of the dataset. This has been done in the code as follows:

```
df.info()
 df.describe()
```

The .info() function provides the number of rows and columns in the dataset, the data types, and the presence of missing values in the dataset.

The .describe() function provides the summary statistics of the dataset, including the mean, minimum, and maximum for the numerical variables in the dataset.

After that, the target variable has been visualized as follows:

```
sns.countplot(x='y', data=df)
 plt.title("Target Variable Distribution")
 plt.show()
```

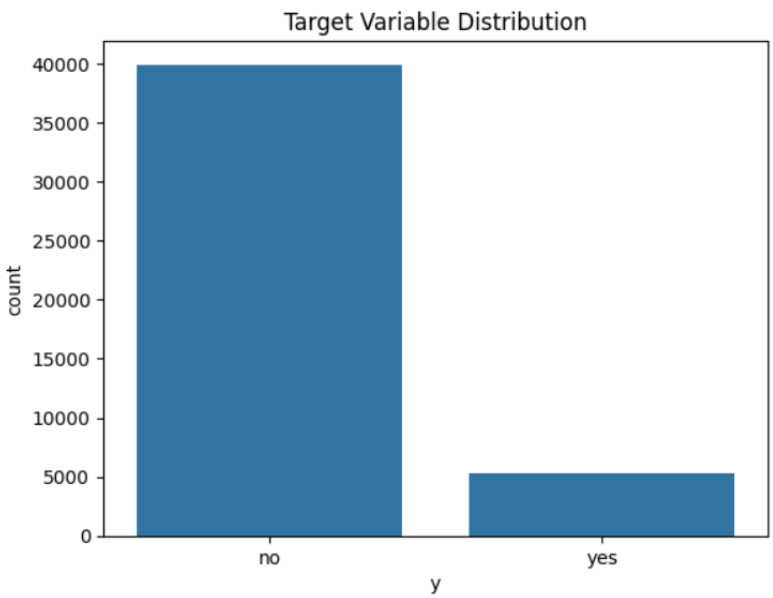

Figure 1: Target Variable Distribution

This graph shows the number of customers who have responded ‘yes’ and the number of customers who have responded ‘no’. We can also verify whether the dataset has class imbalance or not.

If the dataset has a large number of samples in one class relative to the other, it is known as class imbalance in the dataset.

Class imbalance in the dataset may affect the quality of the forecast produced by the machine learning model since it may be biased towards the class with a large number of samples.

Data Preprocessing

Before we train our model, we need to preprocess our data.

1. Converting Target Variable

We have our target variable y with values “yes” and “no”. The model doesn't understand these. So we convert them into numbers.

```
df['y'] = df['y'].map({'yes':1, 'no':0})
```

This means:

```
yes -> 1

no -> 0
```

2. Handling Categorical Variables

We have columns such as job and marital status with values “yes” and “no”. The model doesn't understand these. So we convert them into numbers. (James, et al., 2021)

```
df_encoded = pd.get_dummies(df, drop_first=True)
```

3. Splitting Features and Target

We split our data into features and target.

```
X = df_encoded.drop('y', axis=1)
 y = df_encoded['y']
```

Here,

```
X holds our features

y holds our target
```

So, data preprocessing includes:

```
Converting text to numbers

Handling categorical data

Splitting data into features and target
```

We have now preprocessed our data. The model is now able to learn from it.

Train-Test Split

We split our data into test data and train data.

```
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
 )
```

We have now split our data into test data and train data. The test size is set to 0.2. This means we'll be using 20% for testing and 80% for training.

In simple terms, we've just separated our data into test data and train data.

Out of 100 rows
80 are used to train the model
20 are used to test it

The training data is used to help the model learn. The testing data is used to check how well the model is performing.

The random_state=42 ensures that this split remains constant every time we run this code.

This step is important because it ensures that the model does not “memorize” the data. It also ensures that we can check if it can make predictions on unseen data.

Understanding Entropy

Before we can train our decision tree model, we need to understand entropy. This is because entropy is used by the model to make decisions about splitting the data.

Entropy is used to measure how “mixed” something is (Quinlan, 1986). If something is pure and only contains one kind of item, it has low entropy. If something is “mixed” and contains both kinds of items, it has high entropy.

In this code, we define entropy as follows:

```
def entropy(p):
 return -p*np.log2(p) - (1-p)*np.log2(1-p)
```

Then we plot it using:

```
p = np.linspace(0.01, 0.99, 100)
 ent = entropy(p)

plt.plot(p, ent)
 plt.title("Entropy Curve")
 plt.xlabel("Probability")
 plt.ylabel("Entropy")
 plt.show()
```

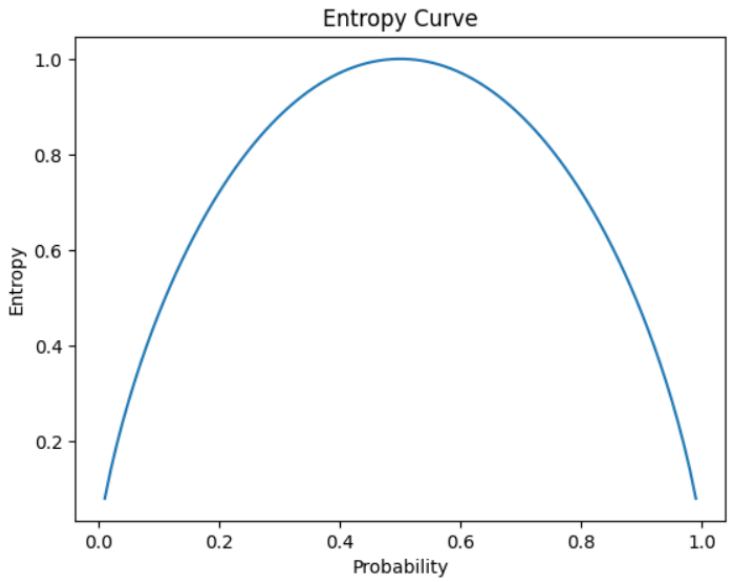

Figure 2: Entropy Curve

From this graph, we can see that:

Entropy is highest when we are at 0.5
Entropy is lowest when we are at 0 or 1

This means that if we have balanced classes in our dataset, we have a high entropy. This also means that if we have pure classes in our dataset, we have low entropy.

Entropy is used by a decision tree model to make decisions about splitting. A decision tree model uses entropy to find the best split. This is done by using information gain.

In information gain, we find features that reduce entropy.

So, entropy is used to help the model make better predictions.

Decision Tree Model Training

We can now prepare our dataset and understand entropy. This means that we can now train our decision tree model using:

```
model = DecisionTreeClassifier(max_depth=5, random_state=42)
 model.fit(X_train, y_train)
```

In the above statement, DecisionTreeClassifier is used to create the model, and fit() function is used to train the model on the dataset (Pedregosa, 2011).

The max_depth=5 parameter controls the depth of the tree that the DecisionTreeClassifier will create. If the tree has a large depth, it will be able to learn more about the dataset, but it may also end up memorizing the dataset, a phenomenon known as overfitting.

By limiting the depth of the tree, the model will remain simple and not end up getting too complicated, which will in turn make it perform well on the new dataset.

The random_state=42 ensures that the results are the same every time the program is run.

During the fitting of the model, the rules are generated by splitting the dataset according to the features present in the dataset (Scikit-learn, 2024). The split that has the most entropy reduction is the one that the model will make at every step.

After the above step, the model is ready to be used to make predictions on the dataset according to the rules it has learned so far.

Predictions

After the model has been trained, it is used to make predictions on the dataset by the following statement:

```
y_pred = model.predict(X_test)
```

The model uses the test dataset (X_test) and makes a prediction on the output by going through the rules learned in the last step and follows the decision tree until it reaches a conclusion (0 or 1) and stores the result in the y_pred variable.

This step is important since it illustrates how the model performs on new, unseen data and also illustrates how the model would be used in a real-world environment.

Model Evaluation

After the predictions have been made by the model, the evaluation of the model's performance is done by calculating the accuracy, the classification report, and the confusion matrix.

1. Accuracy

The accuracy of the model is calculated by the following statement:

```
accuracy = accuracy_score(y_test, y_pred)
 print("Accuracy:", accuracy)
```

The Accuracy metric displays the proportion of accurate predictions made by the model. For instance, if the Accuracy is 0.85, it means 85% of the predictions made by the model are accurate. However, this metric alone may not be sufficient, especially in cases where the data is unbalanced.

2. Classification Report

The following code is used:

```
print(classification_report(y_test, y_pred))
```

This metric provides further information about the performance of the model (James, et al., 2021). The metrics displayed include:

Precision, Recall, F1 Score

This provides further information about the performance of the model.

3. Confusion Matrix

The following code is used:

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
 plt.title("Confusion Matrix")
 plt.xlabel("Predicted")
 plt.ylabel("Actual")
 plt.show()
```

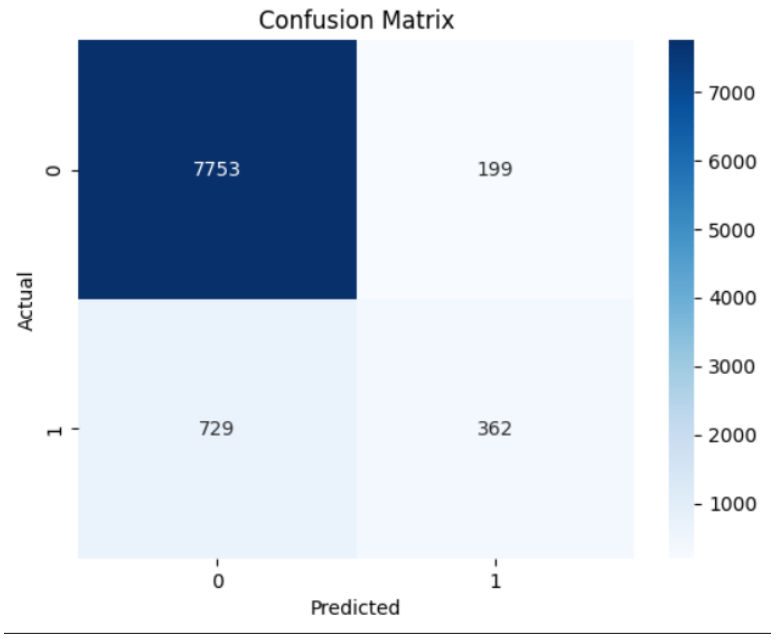

Figure 3: Confusion Matrix

The Confusion Matrix displays the accurate predictions made by the model as well as the predictions made by the model that were not accurate (Powers, 2011). The accurate predictions are displayed on the diagonal line, whereas the inaccurate predictions are displayed in the areas not on the diagonal line.

Decision Tree Visualization

The best part about the decision tree algorithm is that it can easily be visualized. In the code, the following command is used:

```
plt.figure(figsize=(20,10))
 plot_tree(model, filled=True, feature_names=X.columns, class_names=["No", "Yes"])
 plt.show()
```

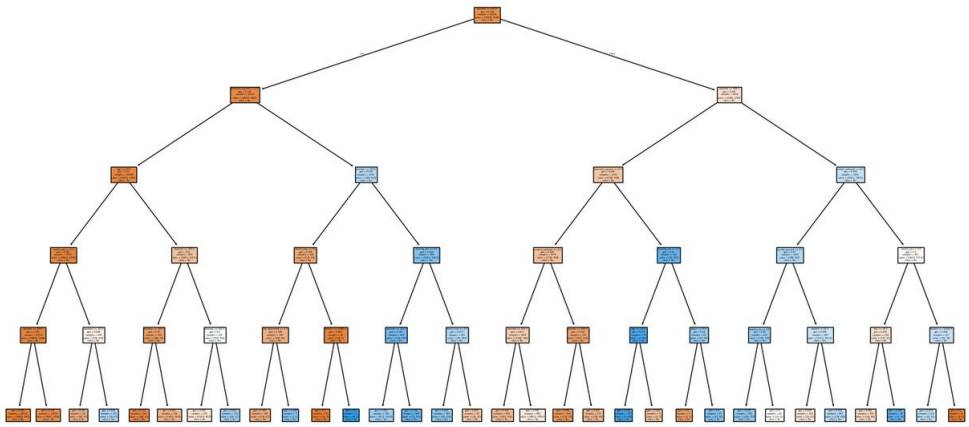

Figure 4: Decision Tree

The above diagram displays how the decisions are made by the model. In the diagram, each box represents a node in the tree, which contains the condition based on the features. Each node splits into two branches depending on the condition:

If the condition is true → go one way

If the condition is false → go the other way

The nodes at the end are the leaf nodes, which contain the prediction made by the model, Yes or No.

The tree also displays further information, including:

i. Number of samples
ii. Class distribution
iii. Entropy

This diagram helps in understanding the decisions made by the model.

Feature Importance

Finally, we can check which are the important features in our model with the following code:

```
importances = model.feature_importances_

feat_imp = pd.Series(importances,
index=X.columns).sort_values(ascending=False)[:10]

feat_imp.plot(kind='barh')
 plt.title("Top 10 Feature Importances")
 plt.show()
```

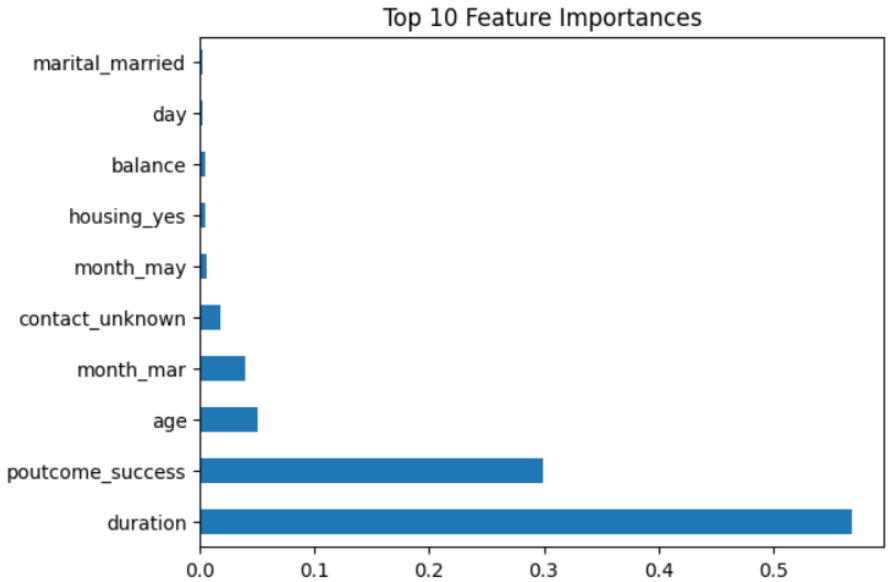

Figure 5: Feature Importance

The above graph shows us the importance of the top features used by our model. The longer the bar, the more important the feature.

The importance of each feature shows us how much each contributes to reducing the entropy during the splitting process. The more important the feature, the more often the decision tree uses it.

This helps us in improving our model because we can now understand which are the factors influencing our model the most. This gives us better insight into our data.

Model Improvement (Depth Testing and Overfitting)

In order to improve our model, we now test how our model performs at different tree depths. We do this in our code with the following line:

depths = [2, 3, 5, 10, 15]

Then we plot our results with the following code:

```
plt.plot(depths, train_acc, label="Train Accuracy")
 plt.plot(depths, test_acc, label="Test Accuracy")
```

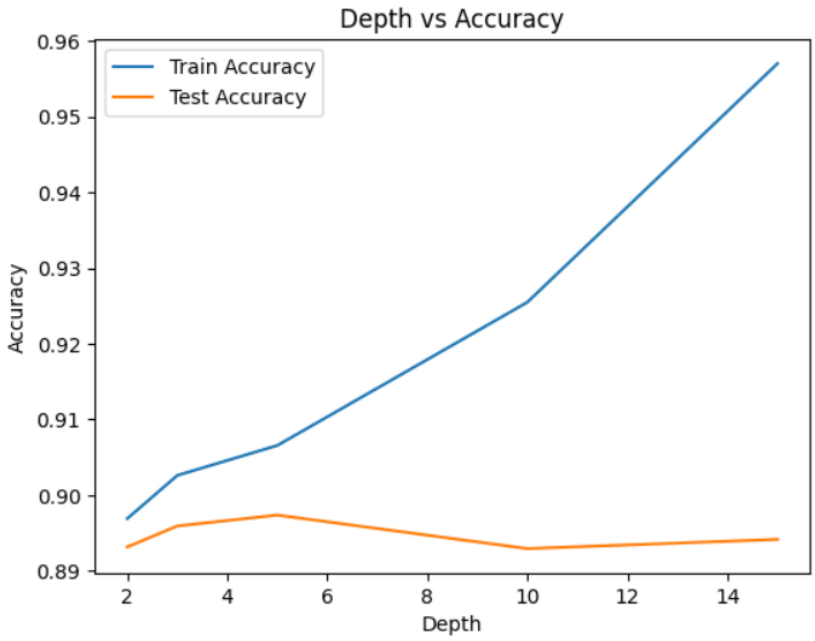

Figure 6: Depth vs Accuracy

The above graph shows us how accuracy changes with the depth of our tree. When the depth is low, our model is not good enough to learn from our data. This phenomenon is called "underfitting" (Goodfellow, et al., 2016).

As we increase our tree's depth, our model starts to improve. But at a certain point, our model starts to get "too good." This phenomenon is called "overfitting." We also plot:

```
plt.plot(depths, train_acc, marker='o')
 plt.plot(depths, test_acc, marker='o')
```

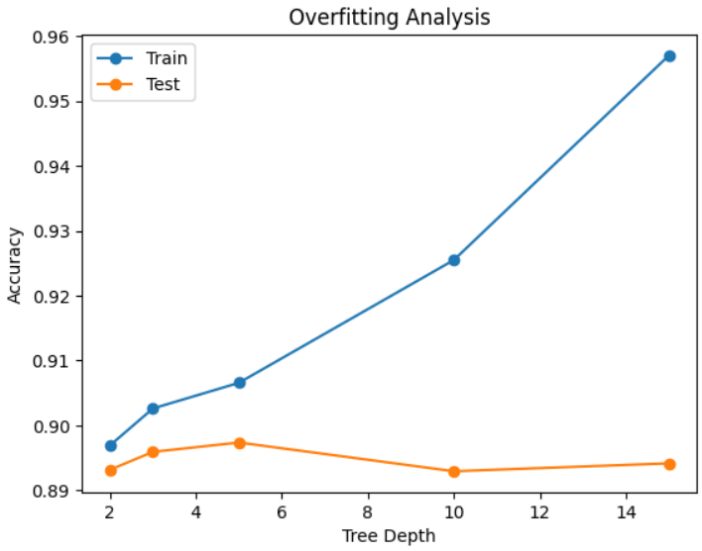

Figure 7: Overfitting Graph

In the above graph, we can see how our model's accuracy increases with our tree's depth. But at the same time, our model might start to "overfit."

So, we select a balanced depth, like 5.

Example Prediction

Next, we make an example prediction using this model.

We do this using the following code:

```
sample = X_test.iloc[0].values.reshape(1, -1)
 prediction = model.predict(sample)

print("Prediction:", prediction)
```

In this code, we make an example prediction using this model.

Here, we take one row from the test data and input it into the model.

The model will make a decision whether it is 0 or 1.

This is how we can use this model in real life.

For any new customer information that we want to input into this model, we can do so.

Discussion

In this project, we used a decision tree model to make a classification. One of the advantages of this model is that it is easy to understand. It is like a flowchart. Each step is like a decision. It can also work with different kinds of data if we encode it. The visualisations also help us understand how this model works. There are also disadvantages. One disadvantage is that this model can overfit.

It can memorize the data if we make it too deep. So, we used max_depth to make sure that it does not overfit. The disadvantage is that this model can also be sensitive to changes.

In all, decision trees are a good and simple approach that has to be well regulated to improve performance.

Conclusion

In this tutorial, we learned how to create a decision tree model step by step using Python. We began by loading and understanding the data set, then learned how to explore and even visualize the target variable. After that, we learned how to prepare the data by transforming text to numbers and encoding the data.

We learned how to divide the data set and create a decision tree model. Then we learned how the model works using entropy and how it decides which feature to use. After that, we learned how to create predictions and even evaluate the model using different metrics.

We learned how to even visualize the decision tree and understand how it works using feature importance. Finally, we learned how to test different depths and understand how to solve underfitting and overfitting issues.

The project has helped us understand how to implement and even understand basic concepts related to decision trees. In all, decision trees are a good and simple approach that can be used for classification.

Important Links

Data Source : https://archive.ics.uci.edu/dataset/222/bank+marketing

Python Code (Google Colab):

https://colab.research.google.com/drive/1LdbwhDo6xpz4d0wA8uWPzd5hGfbS2BIq?usp=sharing

Github Repo: https://github.com/nmhert/machine-learning-tutorial---Decision-tree?tab=readme-ov-file

References

Goodfellow, I., Bengio, Y. & Courville, A., 2016. *Deep Learning*. s.l.:MIT Press.

James, G., Witten, D., Hastie, T. & Tibshirani, R., 2021. *An Introduction to Statistical Learning*. s.l.:Springer.

Mitchell, T., 1997. *Machine Learning*. s.l.:McGraw-Hill.

Pedregosa, F. e. a., 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research,* Volume 12, p. 2825–2830.

Powers, D., 2011. Evaluation: From Precision, Recall and F-measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies,* 2(1), pp. 37-63.

Quinlan, J., 1986. Induction of Decision Trees. *Machine Learning,* 1(1), pp. 81-106.

Repository, U. M. L., 2012. *Bank Marketing Dataset*. [Online]
Available at: https://archive.ics.uci.edu/dataset/222/bank+marketing

Scikit-learn, 2024. *Decision Trees*. [Online]
Available at: https://scikit-learn.org/stable/modules/tree.html